Mini Project Assignment

Topic Area: Robust API Consumption & Error Handling in Python

Student Name	M. Mirudhula Daisy
Register Number	9923005207
Mini Project Topic	Safe Student Result Portal using .get() over Direct Key Access

Scenario

You are building a student result portal that consumes JSON data from a college result API/database export. Not every student record contains every field — some students may be missing an 'attendance' field, some may not yet have a 'grade' field if results are pending, and some optional fields like 'scholarship' only exist for certain students. Accessing a missing key directly with square brackets (e.g., record['grade']) raises a KeyError and crashes the portal.

Problem Statement

Build a student result portal that loads a JSON dataset of student records (some intentionally incomplete) and displays a formatted report card for each student. The program must handle missing keys gracefully using dictionary .get() with sensible defaults, instead of crashing with a KeyError, and must clearly indicate which fields are missing or pending for each student.

Objectives

- Load a JSON dataset containing at least 8 student records with deliberately inconsistent fields.
- Use .get() with default values instead of [] for every optional/uncertain field.
- Demonstrate the KeyError crash first (for comparison), then the fixed safe version.
- Display "N/A" or "Pending" for any missing field on the report card.
- Generate a summary count of how many records had at least one missing field.

Required Features

- JSON dataset with mixed/incomplete student records.
- Two versions of the lookup function: one using [] (buggy) and one using .get() (fixed), for comparison in the report.
- Formatted, per-student report card output (console or simple HTML/PDF).
- Summary statistics of missing-field records.

Suggested Tech Stack

Python, json module, optional pandas for tabular display, optional Flask for a simple web report card view.

Expected Outcome

A result portal that never crashes on incomplete records, clearly marks missing/pending fields, and includes a short before/after comparison showing why .get() is safer than direct key access.

Mini Assessment Questions (to be answered in project report)

- If a JSON key is missing, why might .get() be safer than []?
- What default value did you choose for each missing field, and why?
- Show one example record from your dataset where .get() prevented a crash that [] would have caused.